LAB PRACTICE IV

DEEP LEARNING

SPPU – B.E. Information Technology

Course Code: 414447

Answer-Sheet Writing Guide

All 6 Practical Assignments

Aim • Theory • Algorithm / Procedure • Output • Result

Prepared from the uploaded SPPU Lab Practice IV syllabus

Use this PDF to prepare the written part of your practical examination. Write the program separately on the computer when the examiner asks for implementation.

How to use this guide

For each practical, write the sections below in your answer sheet/journal as required by your examiner: **Aim, Theory, Algorithm/Procedure, Program (if asked in writing), Expected Output, and Result.** The uploaded syllabus lists the six assignments and says practical assessment gives maximum weightage to satisfactory implementation and understanding of the fundamentals. The syllabus does not prescribe one fixed answer-sheet template or one mandatory dataset.

Important: This document gives concise write-ups aligned to the six syllabus assignments. Dataset choice, code, output and exact format may vary according to the faculty's instructions and the dataset available in the lab.

General practical answer format

1. **Aim:** State what you are implementing or studying.

2. **Theory:** Explain the main concept and important components in 4–8 lines.

3. **Algorithm / Procedure:** Write the implementation steps in sequence.

4. **Program:** Write the code only if the examiner asks for a written program; otherwise implement it on the computer.

5. **Output:** Draw or note the actual output produced by your program. Do not invent accuracy values.

6. **Result:** Write one sentence confirming what was successfully implemented.

Practical 1 – Study of Deep Learning Packages

Aim: To study the deep learning packages TensorFlow, Keras, Theano and PyTorch and document their distinct features and functionalities.

Theory

Deep Learning: Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple layers to learn patterns from data.

1. TensorFlow: TensorFlow is an open-source machine learning and deep learning framework developed by Google. It supports tensor operations, automatic differentiation, model training and CPU/GPU execution.

2. Keras: Keras is a high-level deep learning API used to build, train and evaluate neural network models with a simple interface. It is commonly used with TensorFlow.

3. Theano: Theano is a Python library for defining and evaluating mathematical expressions, especially numerical computations involving multidimensional arrays. It supports symbolic computation and automatic differentiation. It is a legacy project.

4. PyTorch: PyTorch is an open-source machine learning framework developed by Meta. It provides tensor computation, automatic differentiation and tools for building and training neural networks.

Distinct features / comparison

Package	Main functionality	Important feature
TensorFlow	Build and train machine learning / deep learning models	Tensor operations, automatic differentiation, CPU/GPU suppor
Keras	Create neural network models using a high-level API	Simple and readable model-building interface
Theano	Symbolic mathematical and numerical computation	Automatic differentiation; legacy package
PyTorch	Tensor computation and deep learning model development	Dynamic computation graphs and autograd

Applications

• Image classification and recognition

• Speech recognition

• Natural Language Processing (NLP)

• Medical image analysis

• Prediction and recommendation systems

Algorithm / Procedure

1. Identify the four deep learning packages.

2. Study the purpose and main features of each package.

3. Compare their functionalities and typical applications.

4. Import the available packages in Python and check their installed versions, if required.

5. Record the observations and comparison in the journal.

Expected observation

The study shows that TensorFlow and PyTorch are deep learning frameworks, Keras is a high-level API, and Theano is a symbolic mathematical computation library.

Result: The deep learning packages TensorFlow, Keras, Theano and PyTorch were studied and their features and functionalities were documented.

Computer demonstration (if asked)

A simple package-import/version-check program may be executed on the computer. Theano may not install on modern Python versions because it is a legacy package; explain this limitation honestly.

Practical 2 – Implementing Feedforward Neural Network with Keras and TensorFlow

Aim: To implement a Feedforward Neural Network using Keras and TensorFlow with a suitable dataset such as MNIST.

Theory

A Feedforward Neural Network (FNN) is an artificial neural network in which information moves from the input layer through one or more hidden layers to the output layer. It does not contain feedback connections.

Layers: Input layer receives data; hidden layers learn patterns using weights, biases and activation functions; output layer produces the prediction.

MNIST: MNIST is a dataset of handwritten digit images from 0 to 9. Each image is 28 × 28 pixels.

Important terms: ReLU is an activation function; Softmax converts output scores into class probabilities; loss measures prediction error; an optimizer updates model weights.

Algorithm / Procedure

1. Import TensorFlow, Keras and plotting libraries.
2. Load the MNIST training and testing datasets.
3. Normalize pixel values by dividing by 255.0.
4. Flatten each 28 × 28 image into a one-dimensional vector.
5. Create a Sequential model with an input/flatten layer, a Dense hidden layer and a 10-neuron output layer.
6. Compile the model using an optimizer, loss function and accuracy metric.
7. Train the model using the training data.
8. Evaluate the model using the test data.
9. Display the test loss and accuracy; plot training accuracy or loss if required.

Model architecture

Layer	Purpose
Flatten (28 × 28)	Converts the image matrix into a one-dimensional vector.
Dense (128, ReLU)	Hidden layer that learns features.
Dense (10, Softmax)	Produces probabilities for ten digit classes (0–9).

Expected output

The program displays epoch-wise training information, followed by test loss and test accuracy. A graph of training accuracy/loss may also be displayed. Actual values depend on the run.

Result: A Feedforward Neural Network was implemented using Keras and TensorFlow and its performance was evaluated on the test dataset.

Important points to remember

• Normalization scales pixel values to the range 0 to 1.

• Epoch means one complete pass through the training dataset.

• Batch size is the number of samples processed before a weight update.

• Sparse categorical cross-entropy is suitable when class labels are integer values.

Practical 3 – Image Classification using CNN

Aim: To build and evaluate an image classification model using a Convolutional Neural Network (CNN).

Theory

A Convolutional Neural Network is a deep learning model commonly used for image-related tasks. It learns spatial features such as edges, textures and shapes from images.

Convolution layer: Applies filters to an image to extract features. **Activation (ReLU):** Adds non-linearity. **Pooling:** Reduces the spatial dimensions of feature maps. **Flatten:** Converts feature maps into a vector. **Dense layer:** Performs classification.

Algorithm / Procedure

1. Import the required libraries.
2. Load a suitable image dataset, for example MNIST or CIFAR-10.
3. Preprocess the images: resize if required, normalize pixel values and arrange the channel dimension.
4. Define the CNN architecture using convolution, activation, pooling, flatten and dense layers.
5. Compile the model with an optimizer, suitable loss function and accuracy metric.
6. Train the model using training images and labels.
7. Evaluate the model using test images and labels.
8. Record the test loss and accuracy.

Basic CNN architecture

Layer	Function
Input	Receives image data.
Conv2D	Extracts local image features using filters.
ReLU	Introduces non-linearity.
MaxPooling2D	Reduces feature-map dimensions.
Flatten	Converts feature maps into a vector.
Dense + Softmax	Classifies the image into output categories.

Expected output

The model displays training progress and test evaluation results. It produces a predicted class for an input image. Accuracy depends on the dataset, model and training settings.

Result: A CNN-based image classification model was built, trained and evaluated.

Additional theory: AlexNet

AlexNet is a well-known CNN architecture that achieved strong results in large-scale image classification. It uses convolutional layers, pooling and fully connected layers.

Practical 4 – Autoencoder for Anomaly Detection

Aim: To use an Autoencoder model for anomaly detection by learning to reconstruct input data.

Theory

An Autoencoder is a neural network that learns a compressed representation of input data and attempts to reconstruct the original input.

Encoder: Converts input into a compact representation. **Latent representation:** The compressed code. **Decoder:** Reconstructs the input from the latent representation.

Anomaly detection: Identifies data samples that differ significantly from normal patterns. An autoencoder can be trained on normal data; samples with high reconstruction error may be treated as anomalies after selecting a suitable threshold.

Mean Squared Error (MSE): Measures the average squared difference between original and reconstructed values.

Algorithm / Procedure

1. Import the required libraries.
2. Load or access a suitable dataset.
3. Preprocess and normalize the data.
4. Select normal samples for training if using a normal-only anomaly detection approach.
5. Build an encoder to compress the input into a latent representation.
6. Build a decoder to reconstruct the original input.
7. Combine encoder and decoder to form the autoencoder.
8. Compile the model using an optimizer and reconstruction loss such as MSE.
9. Train the model by using the input as both input data and target data.
10. Reconstruct test samples and calculate reconstruction error.
11. Choose a threshold and flag samples with error above the threshold as possible anomalies.

Architecture

Component	Function
Input	Original data sample.
Encoder	Compresses the input.
Latent space	Stores the compressed representation.
Decoder	Reconstructs the input.
Reconstruction error	Measures the difference between input and reconstruction.

Expected output

The program displays training loss and reconstruction errors. Samples whose reconstruction error is greater than the chosen threshold are marked as possible anomalies.

Result: An Autoencoder was implemented to reconstruct input data and identify possible anomalies using reconstruction error.

Note: A high reconstruction error indicates a possible anomaly; the threshold must be selected using validation data or a justified method.

Practical 5 – Implementing Continuous Bag of Words (CBOW) Model

Aim: To implement the Continuous Bag of Words (CBOW) model for predicting a target word using surrounding context words.

Theory

CBOW is a Word2Vec architecture used to learn word representations. It predicts a target word from the surrounding context words in a sentence.

Example: In the sentence “I love deep learning”, if “deep” is the target word, the surrounding words can be used as context to predict it.

Tokenization: Splitting text into tokens. **Vocabulary:** The set of unique words. **Word embedding:** A dense vector representation of a word.

Algorithm / Procedure

1. Prepare a text corpus.
2. Convert text to lowercase and tokenize it into words.
3. Create a vocabulary and assign an integer ID to each word.
4. Choose a context-window size.
5. Generate context-target training pairs from the token sequence.
6. Build a neural network with an embedding layer and an output layer.
7. Compile the model using an optimizer and categorical or sparse categorical cross-entropy loss, as appropriate.
8. Train the model on the generated pairs.
9. Use context words as input and predict the target word.

CBOW working

Input	Output
Surrounding context words	Predicted target word

Expected output

The program displays that the CBOW model has been trained and may display the predicted target word for a context input. Prediction quality depends on corpus size, vocabulary and training.

Result: The CBOW model was implemented to predict a target word using its surrounding context words.

CBOW vs Skip-Gram

CBOW	Skip-Gram
Predicts the target word from context words.	Predicts context words from a target word.
Often faster to train.	Can be useful for learning representations of rare words.

Practical 6 – Object Detection using Transfer Learning of CNN Architectures

Aim: To study and implement transfer learning using a pre-trained CNN architecture for an image recognition task.

Theory

Transfer Learning is a technique in which knowledge learned by a model on one dataset or task is reused for a related new task. A pre-trained CNN, such as MobileNetV2, can be used as a feature extractor and a new classifier can be added.

Pre-trained model: A model whose weights have already been learned from a dataset. **Freezing:** Keeping selected weights unchanged during training. **Fine-tuning:** Unfreezing selected pre-trained layers and training them with a small learning rate.

Algorithm / Procedure (as listed in the SPPU syllabus)

1. Load a pre-trained CNN model trained on a large dataset.
2. Freeze parameters (weights) in the model's lower convolutional layers.
3. Add a custom classifier with trainable parameters to the model.
4. Train the classifier layers on the available task-specific training data.
5. Fine-tune hyperparameters and unfreeze more layers as needed.

Expected output

The program displays training and validation metrics such as loss and accuracy. It can predict the class of an input image from the classes used for training.

Result: Transfer learning using a pre-trained CNN architecture was studied/implemented and the model was evaluated on the task-specific dataset.

Important syllabus clarification: The uploaded SPPU syllabus names this assignment "Object detection using Transfer Learning of CNN architectures". Its listed steps describe loading a pre-trained CNN, freezing layers, adding a custom classifier and training it. These steps describe transfer-learning-based image classification; bounding-box object detection requires an object-detection model and bounding-box annotations.

Key points

• MobileNetV2 is a lightweight CNN architecture.

• ImageNet is a large image dataset used to train many pre-trained image models.

• Freezing layers preserves previously learned features during initial training.

• Fine-tuning adapts selected pre-trained weights to the new dataset.

Final Viva Revision – Common Questions

Question	Short answer
What is Deep Learning?	A subset of Machine Learning based on multi-layer neural networks.
What is TensorFlow?	An open-source machine learning and deep learning framework.
What is Keras?	A high-level API for building and training neural networks.
What is PyTorch?	An open-source deep learning framework with tensor and autograd support.
What is CNN?	A neural network commonly used for image data.
What is an epoch?	One complete pass through the training dataset.
What is batch size?	Number of samples processed before a model update.
What is an optimizer?	An algorithm that updates model parameters to reduce loss.
What is a loss function?	A function that measures prediction or reconstruction error.
What is an autoencoder?	A network that learns to encode and reconstruct input data.
What is CBOW?	A Word2Vec model that predicts a target word from context words.
What is transfer learning?	Reusing a pre-trained model for a related task.
What is fine-tuning?	Training selected pre-trained layers on new task data.

Before submitting / finishing the practical

• Write the actual output produced by your own program.

• Do not write a made-up accuracy value.

• Be ready to explain the dataset, layers, loss function and optimizer used.

• Keep your practical journal complete and follow the faculty's specified format.

• Carry college ID card and hall ticket; follow the reporting time and dress code in the exam notice.

Source basis: Uploaded SPPU B.E. Information Technology (2019 Course) syllabus, Lab Practice IV section, pages 44–46 (PDF pages 4–6).